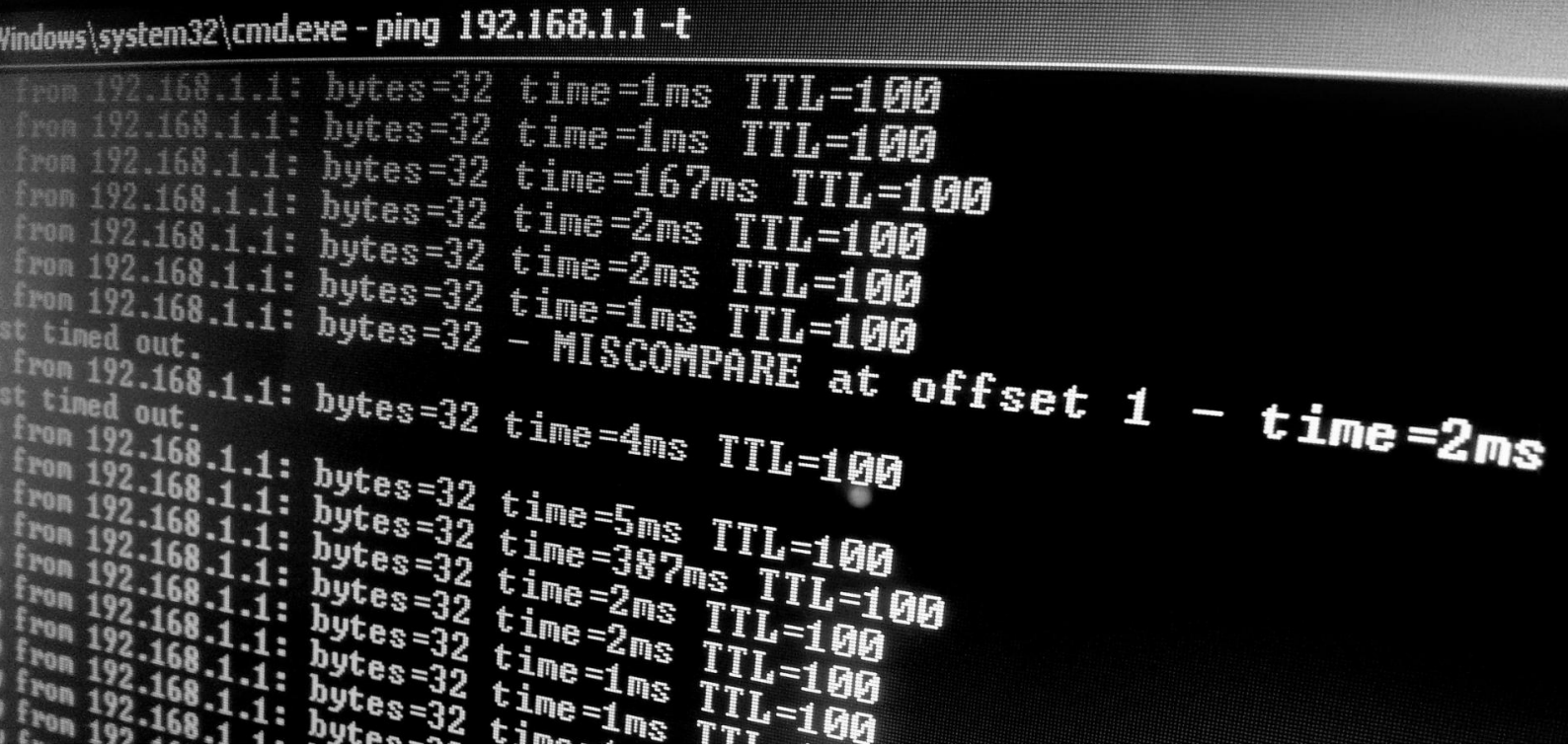


TP2 - Reconnaissance, balayage et débordement de tampon

Travail présenté par BOUCHARD, Charlie (20077301) et SÉGUIN, Charles

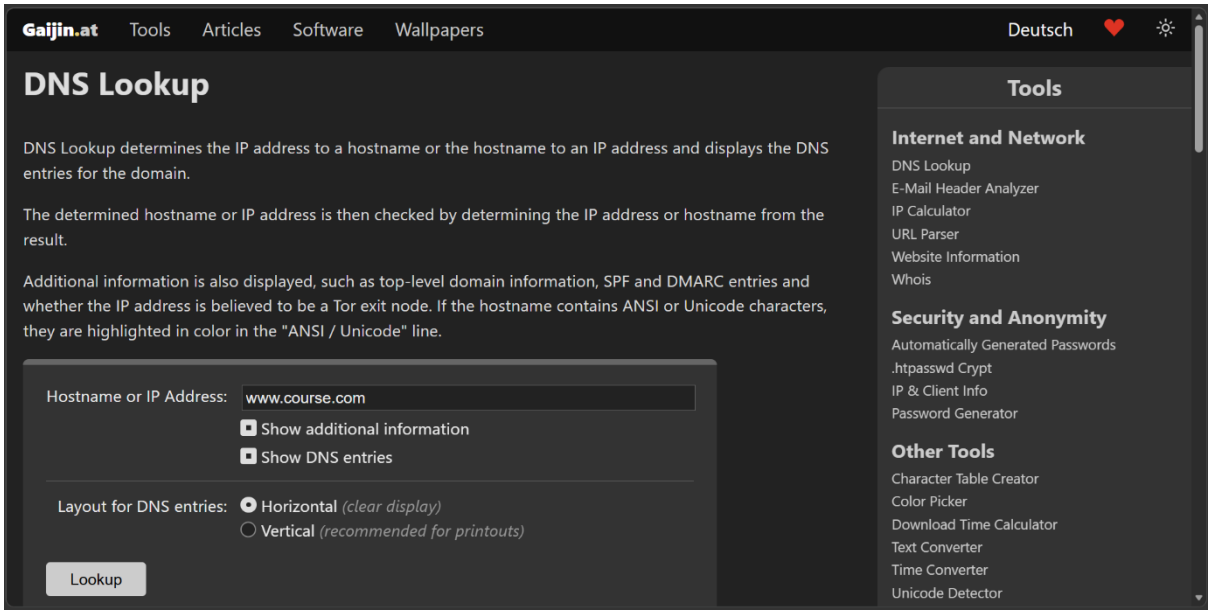
Dans le cadre du cours IFT2830 - Sécurité systèmes informatiques

À la session d’hiver 2026

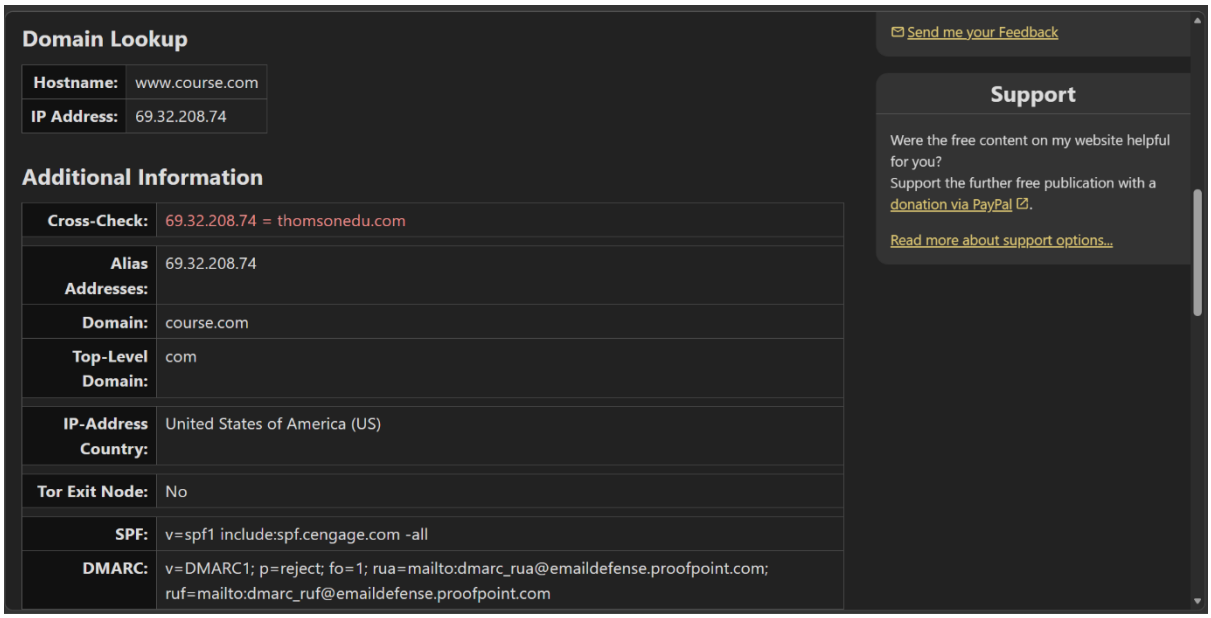
EXERCICE 1 - RECONNAISSANCE WEB

Partie 1 – Outils gaijin

1. Recherche de l'adresse IP de www.course.com

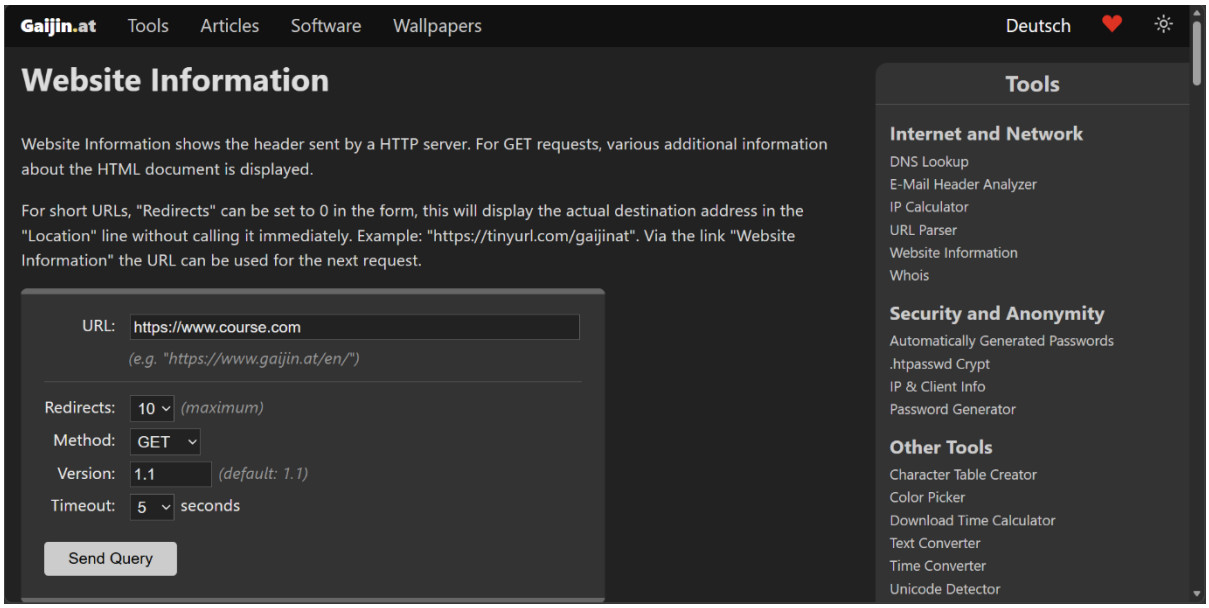


Pour identifier l'adresse IP associée au domaine *www.course.com*, l'outil **DNS Lookup** du site gaijin.at a été utilisé. Cet outil permet d'effectuer une résolution DNS complète et d'afficher les enregistrements associés à un nom de domaine.

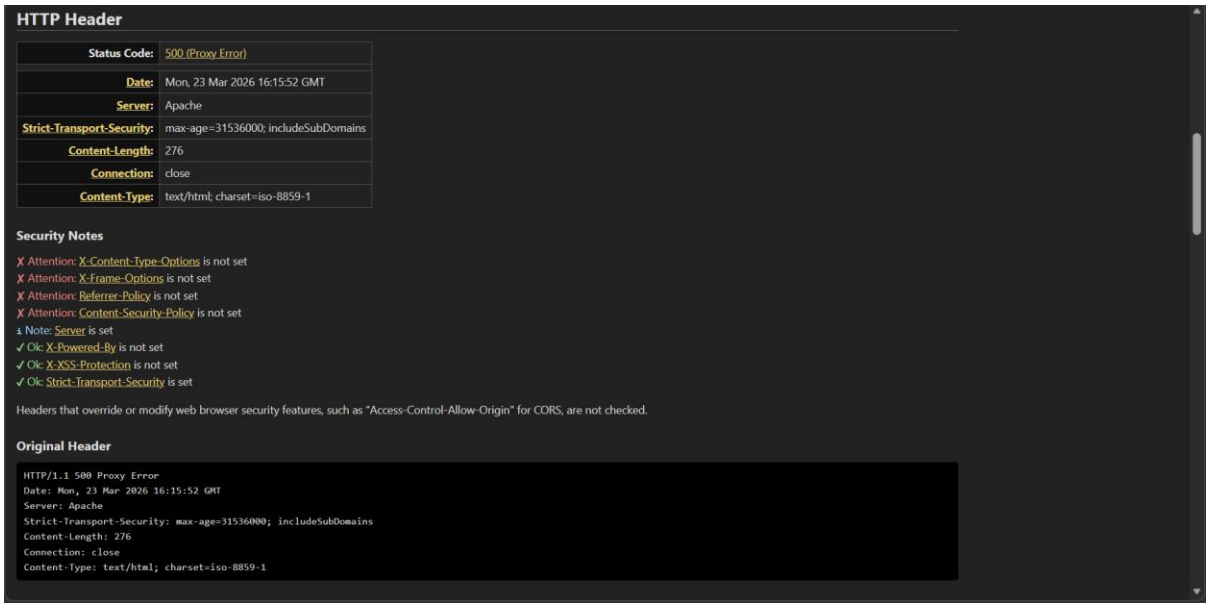


> L'adresse IP retournée pour www.course.com est **69.32.208.74**.
>
> Cette adresse est confirmée par la section *Cross-Check*, qui associe également cette IP au domaine *thomsonedu.com*, ce qui indique une redirection ou un hébergement partagé.

2. Affichage de l'entête HTTP du site Web

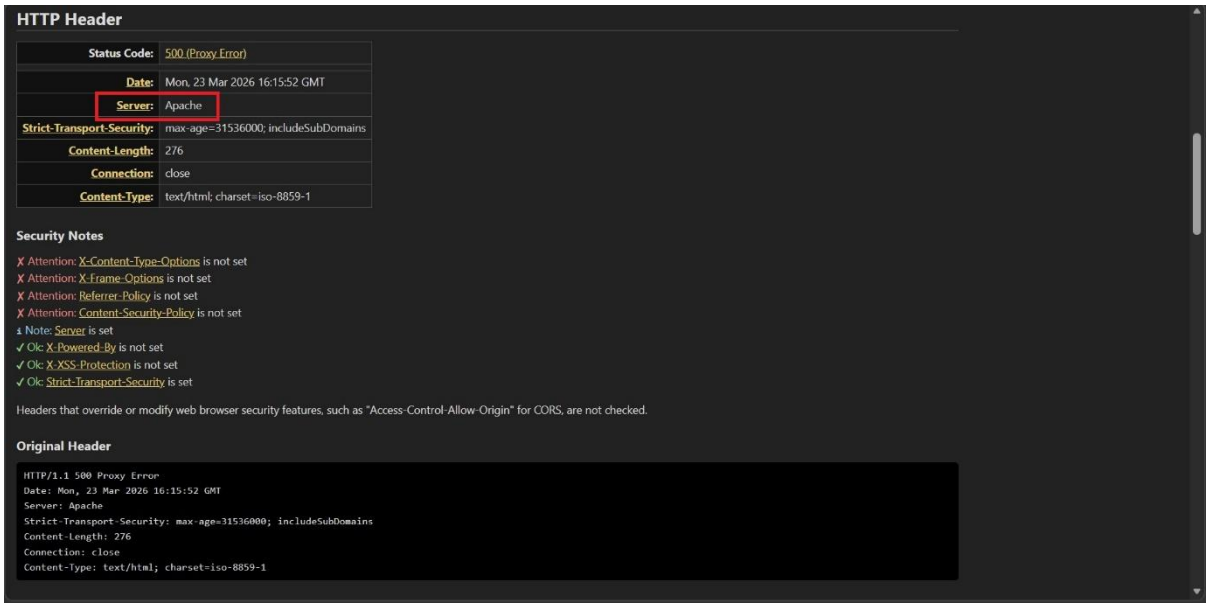


Pour consulter l'entête HTTP renvoyé par le serveur Web de *www.course.com*, l'outil **Website Information** a été utilisé. Cet outil permet d'envoyer une requête HTTP et d'afficher les métadonnées retournées par le serveur.



> Le serveur renvoie un code d'erreur **HTTP 500 (Proxy Error).** L'erreur 500 suggère un problème interne ou une configuration de proxy empêchant l'accès direct au contenu.

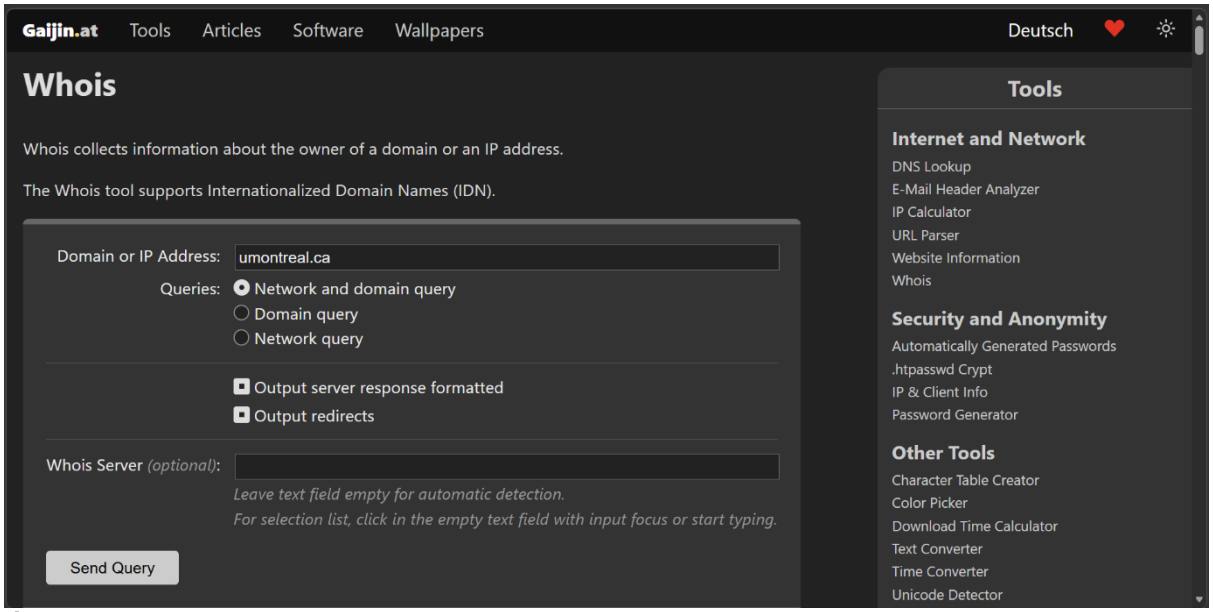
3. Identification du serveur Web



> À partir de l'entête HTTP obtenu à la question précédente, le champ **Server** indique que le site www.course.com est hébergé sur un serveur **Apache**

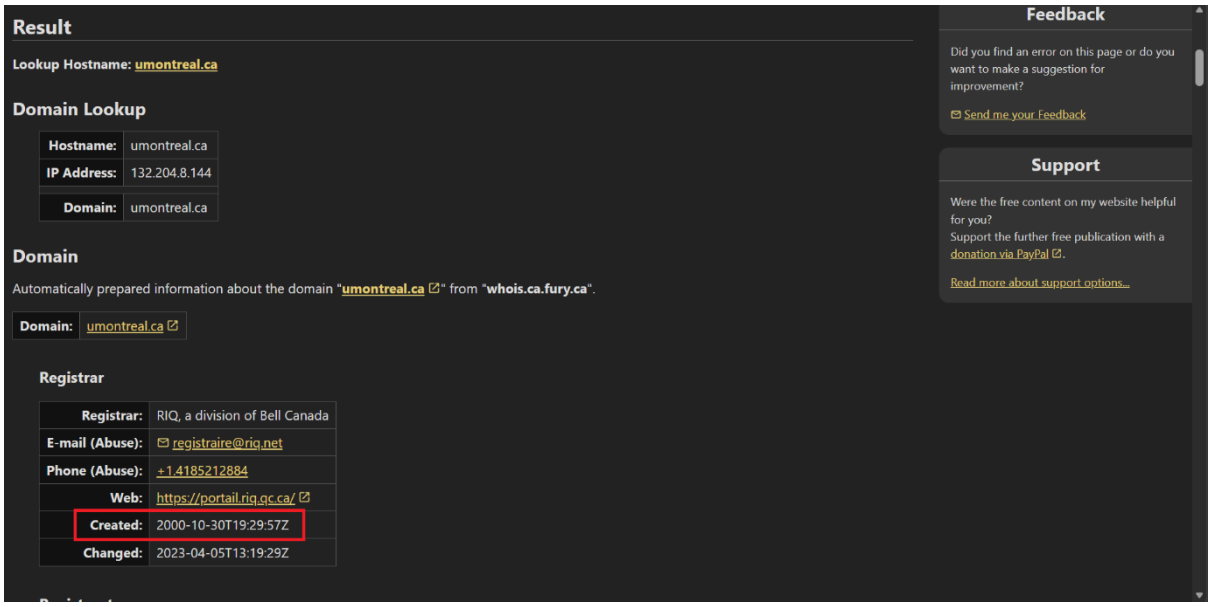
---

4. Obtention des données du domaine umontreal.ca



> L'outil **Whois** de gaijin.at a été utilisé pour obtenir les informations administratives et techniques du domaine *umontreal.ca*. Cet outil interroge automatiquement les serveurs WHOIS pertinents et présente les données de manière structurée.

a. Date de création



Result

Lookup Hostname: umontreal.ca

Domain Lookup

Hostname:	umontreal.ca
IP Address:	132.204.8.144
Domain:	umontreal.ca

Domain

Automatically prepared information about the domain "umontreal.ca" from "whois.ca.fury.ca".

Domain:	umontreal.ca

Registrar

Registrar:	RIQ, a division of Bell Canada
E-mail (Abuse):	registraire@riq.net
Phone (Abuse):	+1.4185212884
Web:	https://portail.riq.qc.ca/
Created:	2000-10-30T19:29:57Z
Changed:	2023-04-05T13:19:29Z

Feedback

Did you find an error on this page or do you want to make a suggestion for improvement?

Send me your Feedback

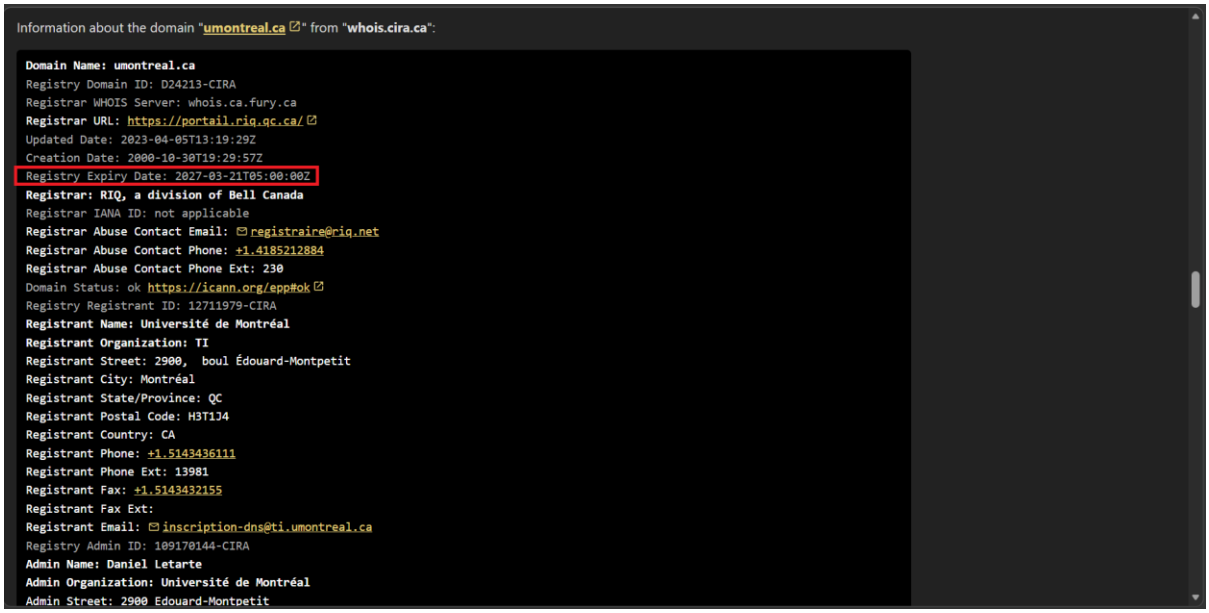
Support

Were the free content on my website helpful for you?
Support the further free publication with a donation via PayPal.

Read more about support options...

> Selon les informations retournées par whois.ca.fury.ca, le domaine umontreal.ca a été créé le : **30 octobre 2000 à 19:29:57 UTC**

b. Date d'expiration

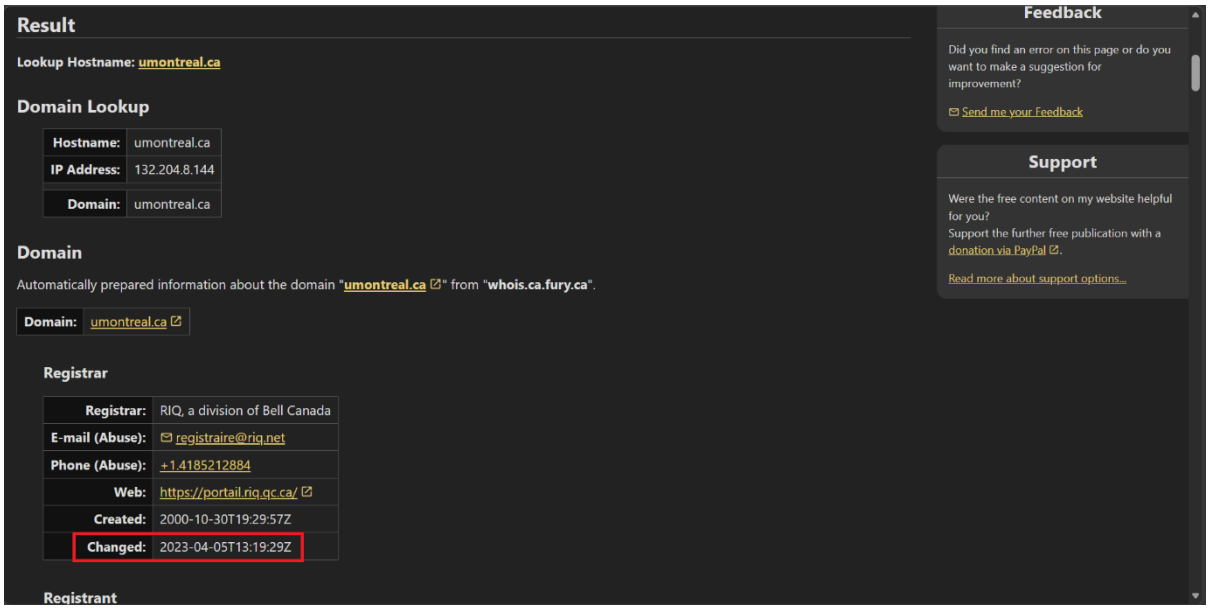


Information about the domain "umontreal.ca" from "whois.cira.ca":

```
Domain Name: umontreal.ca
Registry Domain ID: D24213-CIRA
Registrar WHOIS Server: whois.ca.fury.ca
Registrar URL: https://portail.riq.qc.ca/
Updated Date: 2023-04-05T13:19:29Z
Creation Date: 2000-10-30T19:29:57Z
Registry Expiry Date: 2027-03-21T05:00:00Z
Registrar: RIQ, a division of Bell Canada
Registrar IANA ID: not applicable
Registrar Abuse Contact Email: registraire@riq.net
Registrar Abuse Contact Phone: +1.4185212884
Registrar Abuse Contact Phone Ext: 230
Domain Status: ok https://icann.org/epp#ok
Registry Registrant ID: 12711979-CIRA
Registrant Name: Université de Montréal
Registrant Organization: TI
Registrant Street: 2900,  boul Édouard-Montpetit
Registrant City: Montréal
Registrant State/Province: QC
Registrant Postal Code: H3T1J4
Registrant Country: CA
Registrant Phone: +1.5143436111
Registrant Phone Ext: 13981
Registrant Fax: +1.5143432155
Registrant Fax Ext:
Registrant Email: inscription-dns@ti.umontreal.ca
Registry Admin ID: 109170144-CIRA
Admin Name: Daniel Letarte
Admin Organization: Université de Montréal
Admin Street: 2900 Edouard-Montpetit
```

> Le domaine est enregistré jusqu'au : **21 mars 2027 à 05:00:00 UTC**

c. Date de mise à jour



Result

Lookup Hostname: umontreal.ca

Domain Lookup

Hostname:	umontreal.ca
IP Address:	132.204.8.144
Domain:	umontreal.ca

Domain

Automatically prepared information about the domain "umontreal.ca" from "whois.ca.fury.ca".

Domain:	umontreal.ca

Registrar

Registrar:	RIQ, a division of Bell Canada
E-mail (Abuse):	registraire@riq.net
Phone (Abuse):	+1.4185212884
Web:	https://portail.riq.qc.ca/
Created:	2000-10-30T19:29:57Z
Changed:	2023-04-05T13:19:29Z

Registrant

Feedback

Did you find an error on this page or do you want to make a suggestion for improvement?

Send me your Feedback

Support

Were the free content on my website helpful for you?
Support the further free publication with a donation via PayPal.

Read more about support options...

> La dernière modification enregistrée dans la base WHOIS date du : **5 avril 2023 à 13:19:29 UTC**

d. Nom et coordonnées du contact technique

Technical Contact

Organization:	Université de Montréal
Name:	Daniel Letarte
Address:	2900 Edouard-Montpetit
Postal Code:	H3T1J4
City:	Montreal
State:	QC
Country:	CA (Canada)
E-mail:	inscription-dns@ti.umontreal.ca
Phone:	+1.5143436111
Phone:	38230

Network

Automatically prepared information about the IP address "132.204.8.144" from "whois.arin.net".

Network Range:	132.204.0.0 - 132.204.255.255
Network Range:	132.204.0.0/16

> Le contact technique associé au domaine est :
>
> - Nom : Daniel Letarte
> - Organisation : Université de Montréal
> - Adresse : 2900 ÉdouardMontpetit, Montréal, QC, H3T 1J4, Canada
> - Courriel : inscription-dns@ti.umontreal.ca
> - Téléphone : +1 514 3436111

e. Liste des serveurs de nom (Name Server)

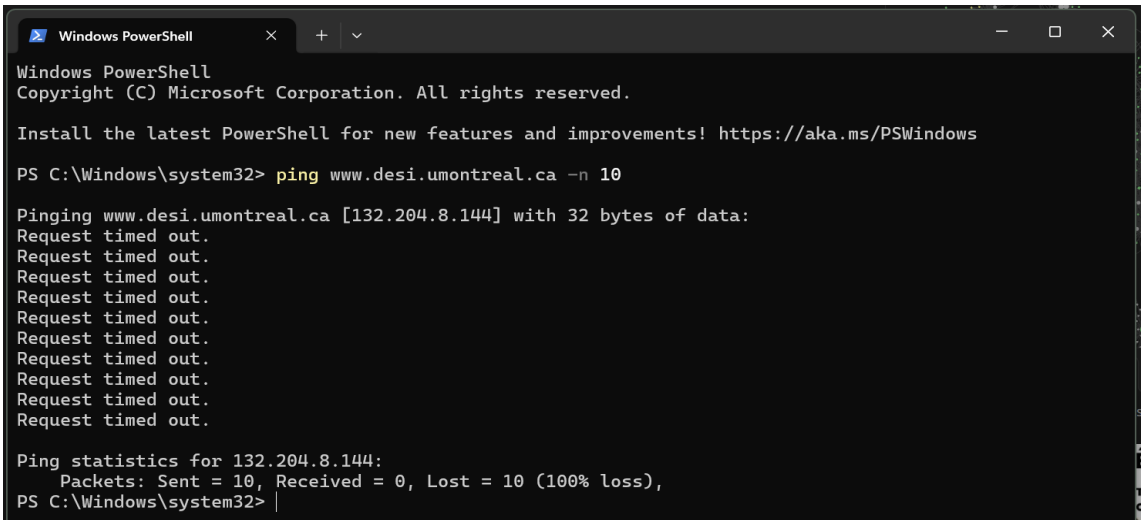
```
Billing Name:
Billing Organization:
Billing Street:
Billing City:
Billing State/Province:
Billing Postal Code:
Billing Country:
Billing Phone:
Billing Phone Ext:
Billing Fax:
Billing Fax Ext:
Billing Email:
Name Server: dnsp1.dit.umontreal.ca
Name Server: dnsp2.dit.umontreal.ca
Name Server: ns1.risq.qc.ca
Name Server: ns2.risq.qc.ca
DNSSEC: unsigned
URL of the ICANN Whois Inaccuracy Complaint Form: https://www.icann.org/wicf/
>>> Last update of WHOIS database: 2026-03-23T16:24:30Z <<<

For more information on Whois status codes, please visit https://icann.org/epp

%
% Use of CIRA's WHOIS service is governed by the Terms of Use in its Legal
% Notice, available at https://www.cira.ca/en/resources/documents/about/website-terms-use
```

> Les serveurs DNS associés au domaine sont :
>
> - **dnsp1.dit.umontreal.ca**
> - **dnsp2.dit.umontreal.ca**
> - **ns1.risq.qc.ca**
> - **ns2.risq.qc.ca**
>
> Ces serveurs assurent la résolution DNS du domaine et garantissent sa redondance.

Partie 2 – Exploration en Terminal

5. Délais des requêtes ping vers desi.umontreal.ca

```
ping www.desi.umontreal.ca -n 10
```



- 10 requêtes envoyées
- 0 réponses
- 100 % de perte
- Request timed out pour chaque tentative

> Les dix requêtes ICMP envoyées vers www.desi.umontreal.ca n'ont généré aucune réponse, ce qui se traduit par une perte totale des paquets.
>
> Ce comportement ne signifie pas nécessairement que le serveur est inactif : il est très probable que le serveur ou le pare-feu institutionnel bloque les requêtes ICMP. Ce type de filtrage est courant dans les environnements universitaires, car il limite la reconnaissance réseau non autorisée et réduit la surface d'attaque.
>
> **L'absence de réponse est donc cohérente avec une politique de sécurité qui désactive volontairement les réponses au ping.**

6. Délais des requêtes ping vers Google.ca

```
ping www.google.ca -n 10
```

```
Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\charl> ping www.google.ca -n 10

Pinging www.google.ca [64.233.178.94] with 32 bytes of data:
Reply from 64.233.178.94: bytes=32 time=5ms TTL=106
Reply from 64.233.178.94: bytes=32 time=4ms TTL=106
Reply from 64.233.178.94: bytes=32 time=5ms TTL=106
Reply from 64.233.178.94: bytes=32 time=5ms TTL=106
Reply from 64.233.178.94: bytes=32 time=5ms TTL=106
Reply from 64.233.178.94: bytes=32 time=5ms TTL=106
Reply from 64.233.178.94: bytes=32 time=5ms TTL=106
Reply from 64.233.178.94: bytes=32 time=5ms TTL=106
Reply from 64.233.178.94: bytes=32 time=5ms TTL=106
Reply from 64.233.178.94: bytes=32 time=5ms TTL=106

Ping statistics for 64.233.178.94:
    Packets: Sent = 10, Received = 10, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 4ms, Maximum = 5ms, Average = 4ms
PS C:\Users\charl>
```

- Adresse IP cible : 64.233.178.94
- 10 paquets envoyés, 10 reçus → 0 % de perte
- Temps de réponse : entre 4 ms et 5 ms
- Moyenne: 4 ms

> Les dix requêtes ICMP envoyées vers www.google.ca ont toutes reçu une réponse, avec une latence extrêmement faible (4 à 5 ms) et aucune perte de paquets.
>
> **Ces résultats témoignent d'une connectivité réseau très performante. Google s'appuie sur une infrastructure mondiale de serveurs CDN, dont plusieurs sont situés à proximité (notamment à Montréal), ce qui explique la rapidité et la stabilité des réponses. Le TTL élevé confirme que le serveur contacté se trouve relativement près dans la topologie réseau.**

7. Analyse du traceroute vers google.ca

```
tracert www.google.ca
```

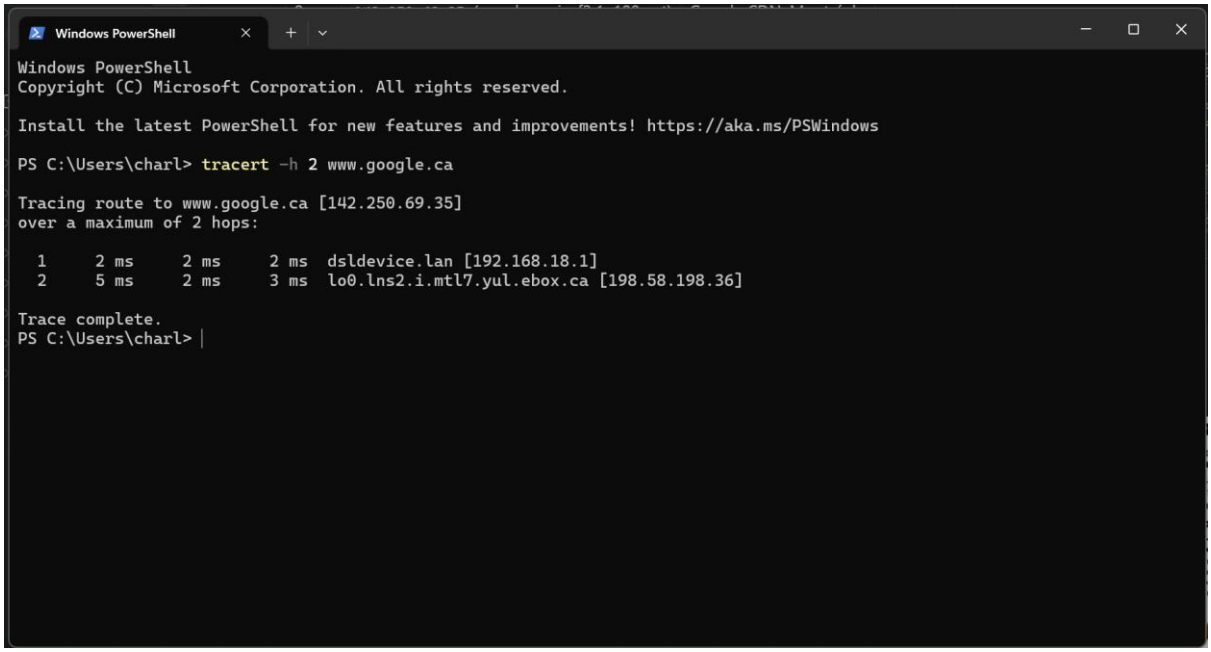
```
Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\charl> tracert www.google.ca

Tracing route to www.google.ca [142.250.69.35]
over a maximum of 30 hops:

  1     3 ms     2 ms     2 ms  dsldevice.lan [192.168.18.1]
  2     5 ms     4 ms     4 ms  lo0.lns2.i.mtl7.yul.ebox.ca [198.58.198.36]
  3     3 ms     3 ms     3 ms  901.ae13.er1.mtl7.yul.ebox.ca [67.221.48.85]
  4     4 ms     3 ms     3 ms  0.ae23.er3.mtl3.yul.ebox.ca [96.127.249.38]
  5     4 ms     3 ms     3 ms  142.250.174.26
  6     6 ms     4 ms     4 ms  142.251.64.219
  7     6 ms     4 ms     4 ms  142.250.238.147
  8     4 ms     3 ms     4 ms  pnyula-aa-in-f3.1e100.net [142.250.69.35]

Trace complete.
PS C:\Users\charl>
```

Hop	Adresse IP / Nom	Localisation probable
1	192.168.18.1 (dsldevice.lan)	Routeur local (Montréal)
2	198.58.198.36 (ebox.ca)	EBOX, Montréal
3	67.221.48.85 (ebox.ca)	EBOX, Montréal
4	96.127.249.38 (ebox.ca)	EBOX, Montréal
5	142.250.174.26	Google, Montréal
6	142.251.64.219	Google, Montréal
7	142.250.238.147	Google, Montréal
8	142.250.69.35 (pnyula-aa-in-f3.1e100.net)	Google CDN, Montréal

> Le traceroute révèle un chemin composé de huit sauts. Les premiers routeurs appartiennent au réseau local et au fournisseur d'accès EBOX, tous situés à Montréal. Les sauts suivants correspondent à l'infrastructure Google, également localisée dans la région, ce qui confirme la présence de serveurs CDN proches de l'utilisateur.
>
> **Cette proximité géographique explique la faible latence observée lors du ping. Le dernier nœud `pnyula-aa-in-f3.1e100.net` correspond aux serveurs de distribution de contenu de Google, optimisés pour la rapidité et la disponibilité.**

8. Impact d'une limite de 2 hops

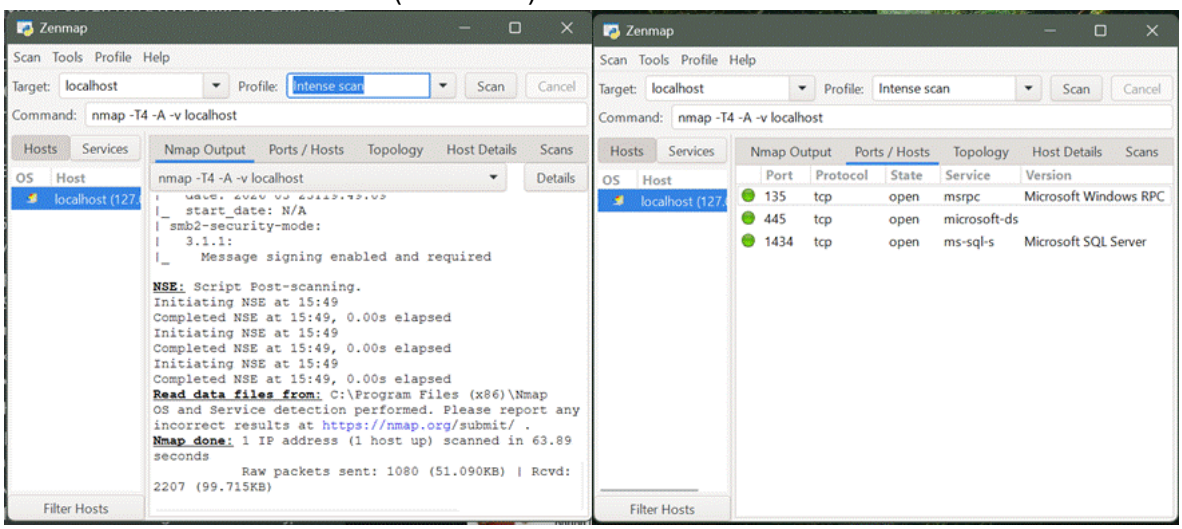
```
tracert -h 2 www.google.ca
```



> Lorsque la limite de sauts est fixée à 2, le traceroute n'a pas la possibilité d'atteindre le serveur cible. Seuls les deux premiers routeurs du chemin apparaissent : le routeur local et le premier routeur du fournisseur d'accès.
>
> Après le deuxième saut, le TTL expire et le paquet est abandonné, ce qui interrompt la trace.
>
> **Ce comportement illustre le fonctionnement du traceroute, qui repose sur un TTL croissant pour découvrir progressivement les routeurs intermédiaires. Un TTL trop faible empêche donc d'atteindre la destination finale**

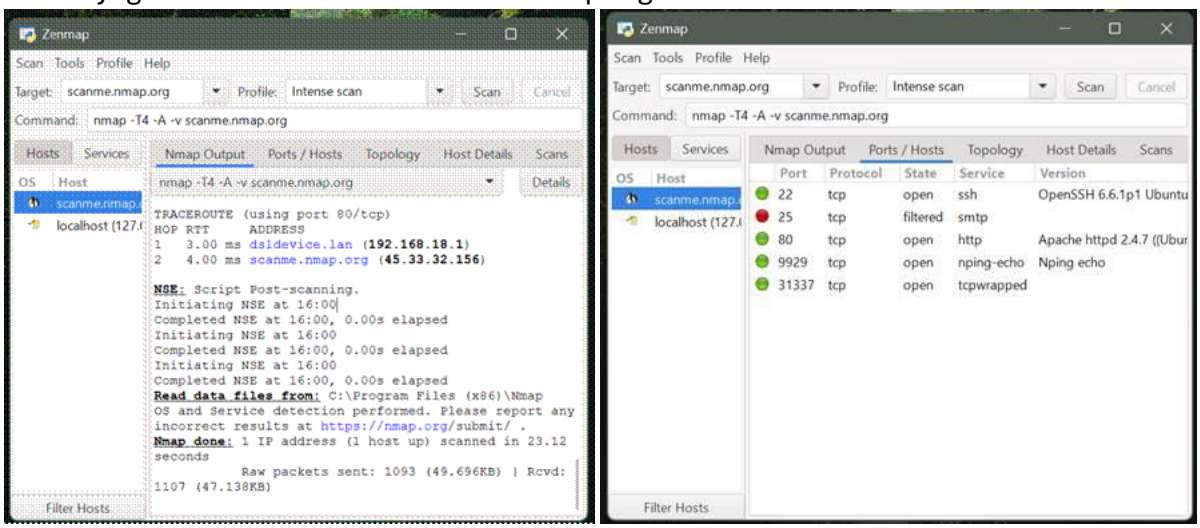
EXERCICE 2 – BALAYAGE

1. Scan de la machine locale (localhost)



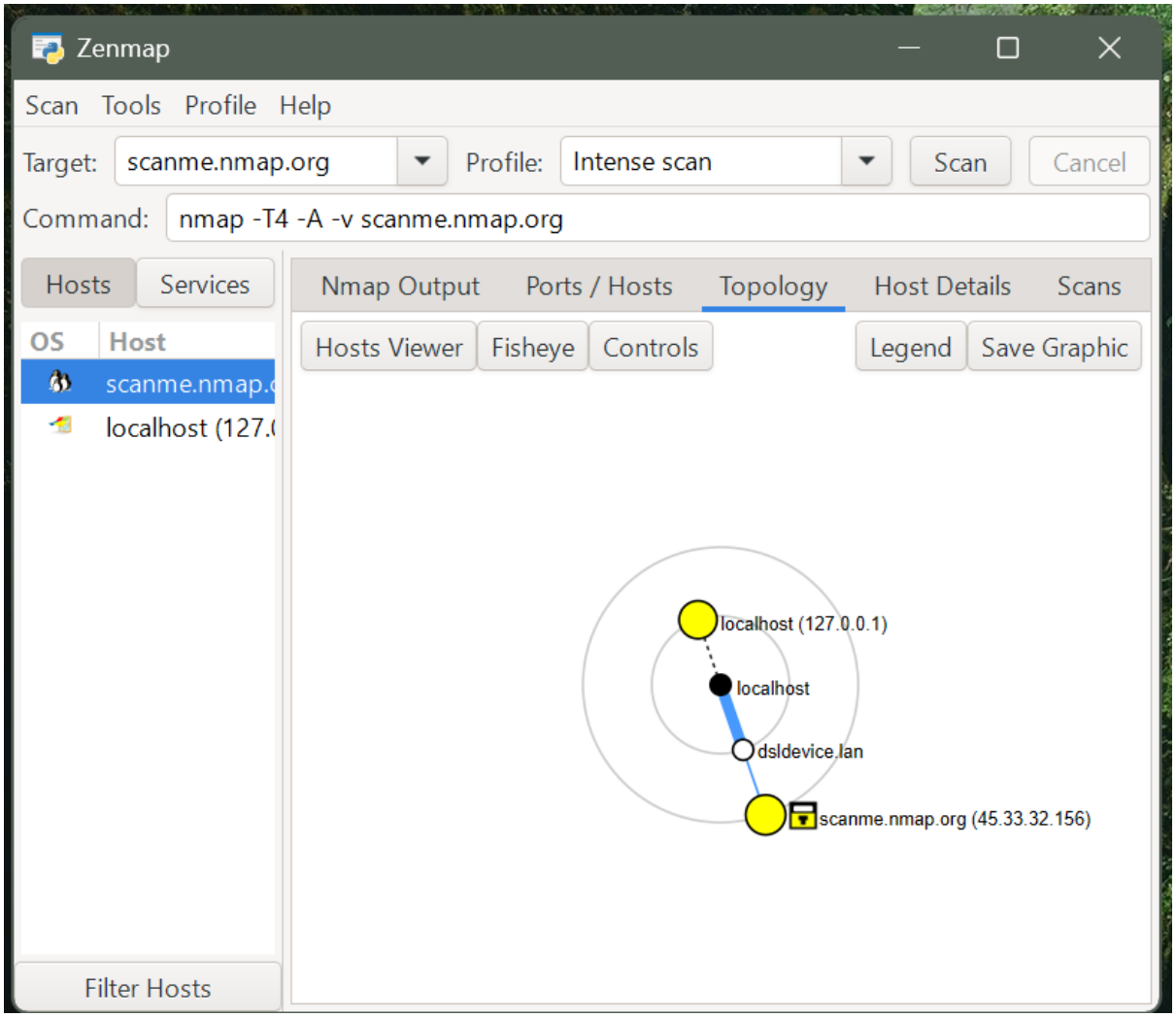
> **Le balayage de la machine locale met en évidence trois ports ouverts correspondant à des services Windows (RPC, SMB et SQL Server Browser).** Leur présence est normale et reflète le fonctionnement interne du système d'exploitation. Ces ports ne sont accessibles qu'en local et ne constituent pas une exposition réseau externe.

2. Balayage du serveur distant scanme.nmap.org



> Le balayage de scanme.nmap.org révèle cinq ports intéressants :
>
> - **Trois ports (22, 80, 9929) sont ouverts et exposent des services légitimes** fournis par l'équipe Nmap pour l'apprentissage du scanning.
> - **Le port 31337 est ouvert mais protégé par TCP Wrapper**, ce qui limite l'information divulguée.
> - **Le port 25 est filtré, ce qui indique la présence d'un pare-feu empêchant Nmap d'en déterminer l'état réel**.
>
> Ces résultats semblent cohérents avec la configuration pédagogique du serveur scanme.nmap.org.

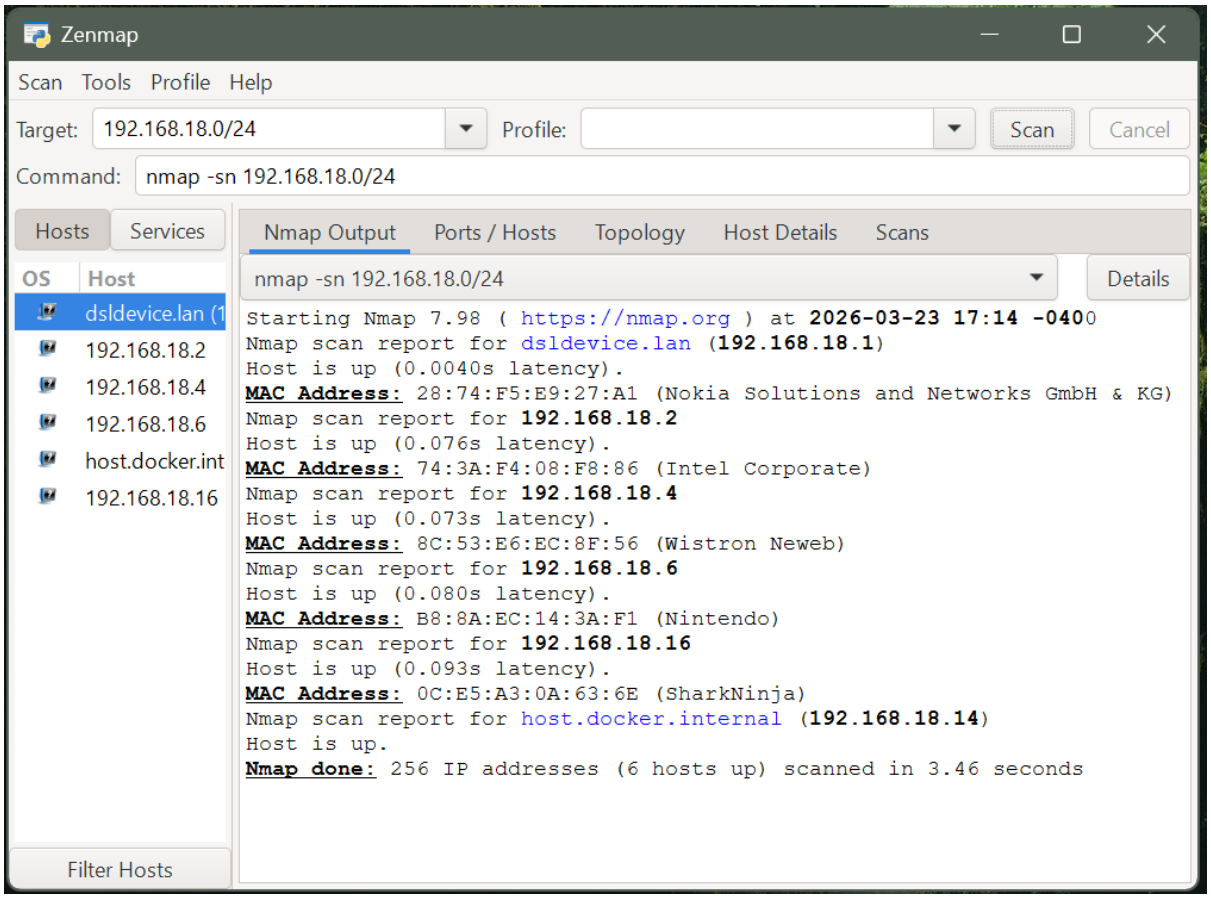
3. Interprétation de l'onglet « Topology »



> La topologie indique que la machine locale atteint le serveur *scanme.nmap.org* en ne traversant qu'un seul routeur intermédiaire, ce qui correspond à une distance réseau de **2 hops**.
>
> Zenmap représente ces nœuds sous forme de graphe, où chaque cercle correspond à un équipement réseau détecté lors du traceroute intégré au scan.
>
> **Cette visualisation confirme que la route vers la cible est directe et que le serveur est accessible sans passer par une longue chaîne de routeurs. La simplicité de la topologie est cohérente avec la faible latence observée (≈ 3–4 ms), ce qui indique une connexion rapide et un chemin réseau court.**

4. Balayage du réseau local

```
nmap -sn 192.168.18.0/24
```



> Le scan Nmap du sous-réseau 192.168.18.0/24 a permis d'identifier **6 machines actives** sur un total de 256 adresses IP analysées.

Hôtes détectés

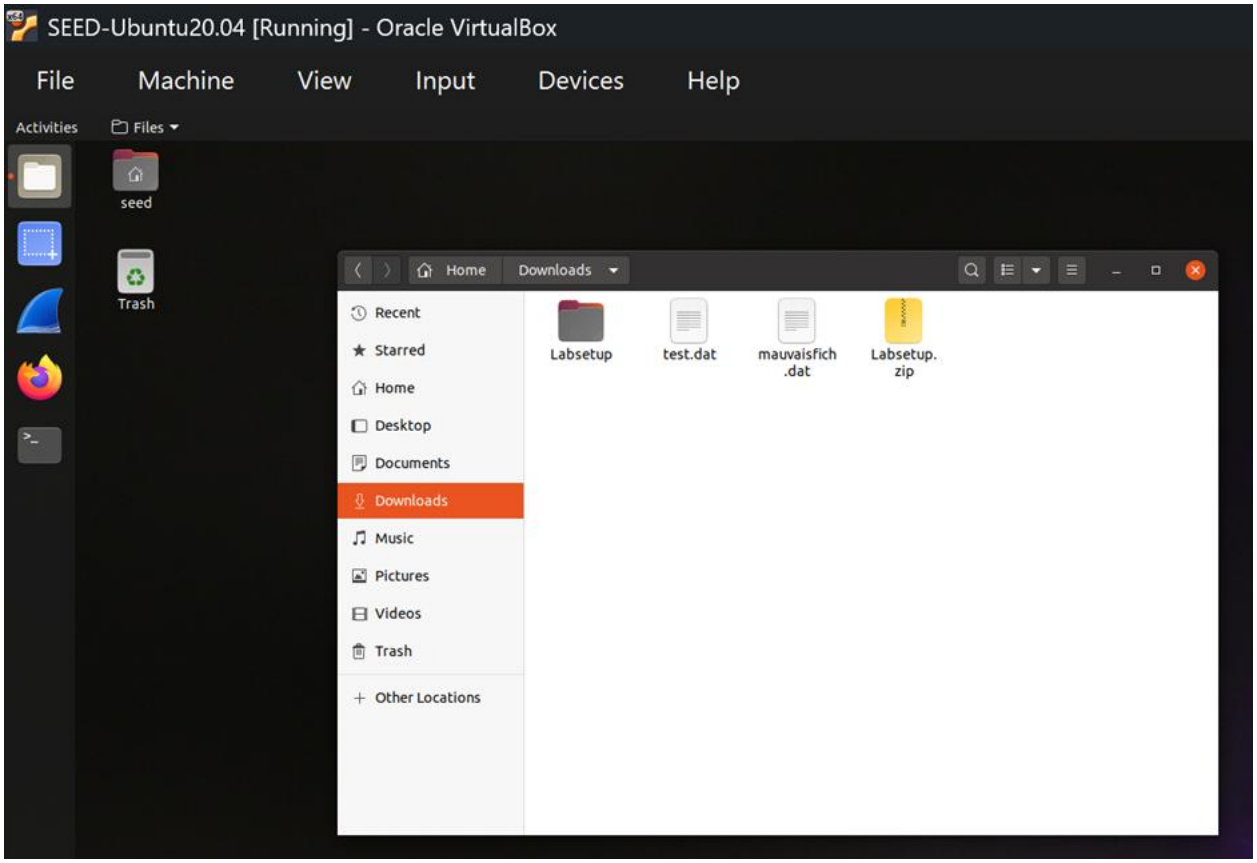
Adresse IP	Nom d'hôte	Statut	Adresse MAC	Fabricant
192.168.18.1	dsldevice.lan	Host is up	28:74:F5:E9:27:A1	Nokia Solutions and Networks
192.168.18.2	—	Host is up	74:3A:F4:08:F8:86	Intel Corporate
192.168.18.4	—	Host is up	8C:53:E6:EC:8F:56	Wistron Neweb
192.168.18.6	—	Host is up	B8:8A:EC:14:3A:F1	Nintendo
192.168.18.16	—	Host is up	0C:E5:A3:0A:63:6E	SharkNinja
192.168.18.14	host.docker.internal	Host is up	(non fourni)	Machine virtuelle / Docker

5. Commande Nmap pour balayer dix serveurs Web

```
nmap –sV www.quebec.ca www.doomsdayclock.net www.kaggle.com www.uptodate.com
www.creativecommons.org www.ruisss.umontreal.ca www.github.com www.puginarug.com
www.timeanddate.com www.myfonts.com
```

EXERCICE 3 – DEBORDEMENT DE TAMPON

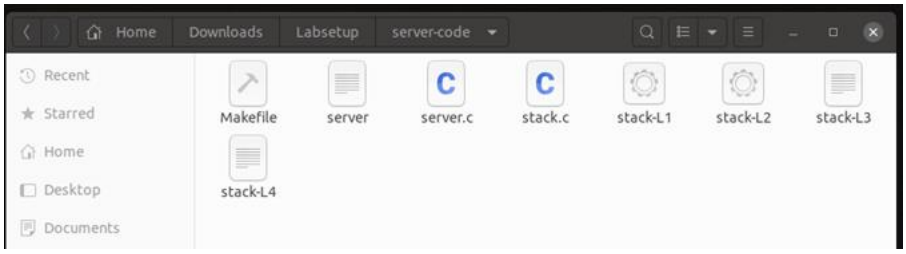
1. Installation de la machine virtuelle



2. Désactivation d'une contremesure

> La commande désactive la contremesure liée à l'ASLR (Address Space Layout Randomization). Cette protection a pour rôle de randomiser l'emplacement des zones mémoire (pile, tas, bibliothèques, etc.) afin d'empêcher un attaquant de prédire où injecter du code ou où détourner l'exécution.
>
> En la désactivant, **les adresses mémoire deviennent prévisibles**, ce qui facilite l'analyse et la reproduction d'exploits basés sur le dépassement de tampon.

3. Contremesure désactivée lors de la compilation



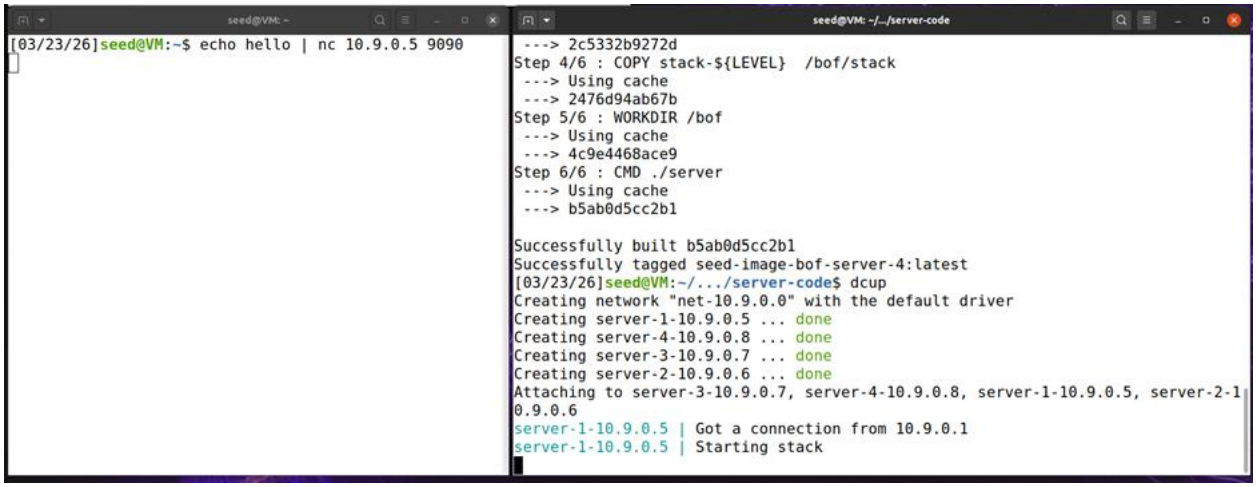

```
seed@VM: ~/.../server-code
[03/23/26]seed@VM:~/.../server-code$ make
gcc -o server server.c
gcc -DBUF_SIZE=100 -DSHOW_FP -z execstack -fno-stack-protector -static -m32 -o s
tack-L1 stack.c
gcc -DBUF_SIZE=180 -z execstack -fno-stack-protector -static -m32 -o stack-L2 st
ack.c
gcc -DBUF_SIZE=200 -DSHOW_FP -z execstack -fno-stack-protector -o stack-L3 stack
.c
gcc -DBUF_SIZE=80 -DSHOW_FP -z execstack -fno-stack-protector -o stack-L4 stack.
c
[03/23/26]seed@VM:~/.../server-code$ make install
cp server ../bof-containers
cp stack-* ../bof-containers
[03/23/26]seed@VM:~/.../server-code$
```

> Les options de compilation utilisées désactivent deux contremesures essentielles de protection de la pile :
>
> - **fno-stack-protector** : désactive les *stack canaries*, un mécanisme qui détecte les débordements de tampon avant qu'ils ne corrompent la pile.
> - **z execstack** : rend la pile **exécutable**, ce qui permet à un attaquant d'y injecter du code et de l'exécuter, alors qu'une pile normale est marquée comme non exécutable (NX bit).
>
> Ensemble, ces options suppriment la détection des débordements et permettent l'exécution de code sur la pile, rendant le programme vulnérable aux attaques classiques de type **buffer overflow**.

4. Exécution des commandes de lancement

```
Step 4/6 : COPY stack-${LEVEL}  /bof/stack
 ---> Using cache
 ---> 2a33ebd865f5
Step 5/6 : WORKDIR /bof
 ---> Using cache
 ---> 3b142507b14f
Step 6/6 : CMD ./server
 ---> Using cache
 ---> 825f284b3d71

Successfully built 825f284b3d71
Successfully tagged seed-image-bof-server-3:latest
Building bof-server-L4
Step 1/6 : FROM handsonsecurity/seed-ubuntu:small
 ---> 1102071f4a1d
Step 2/6 : COPY server    /bof/
 ---> Using cache
 ---> 337966b3f4fb
Step 3/6 : ARG LEVEL
 ---> Using cache
 ---> 2c5332b9272d
Step 4/6 : COPY stack-${LEVEL}  /bof/stack
 ---> Using cache
 ---> 2476d94ab67b
Step 5/6 : WORKDIR /bof
 ---> Using cache
 ---> 4c9e4468ace9
Step 6/6 : CMD ./server
 ---> Using cache
 ---> b5ab0d5cc2b1

Successfully built b5ab0d5cc2b1
Successfully tagged seed-image-bof-server-4:latest
[03/23/26]seed@VM:~/.../server-code$ dcup
Creating network "net-10.9.0.0" with the default driver
Creating server-1-10.9.0.5 ... done
Creating server-4-10.9.0.8 ... done
Creating server-3-10.9.0.7 ... done
Creating server-2-10.9.0.6 ... done
Attaching to server-3-10.9.0.7, server-4-10.9.0.8, server-1-10.9.0.5, server-2-1
0.9.0.6
```

5. Analyse du comportement dans la première fenêtre

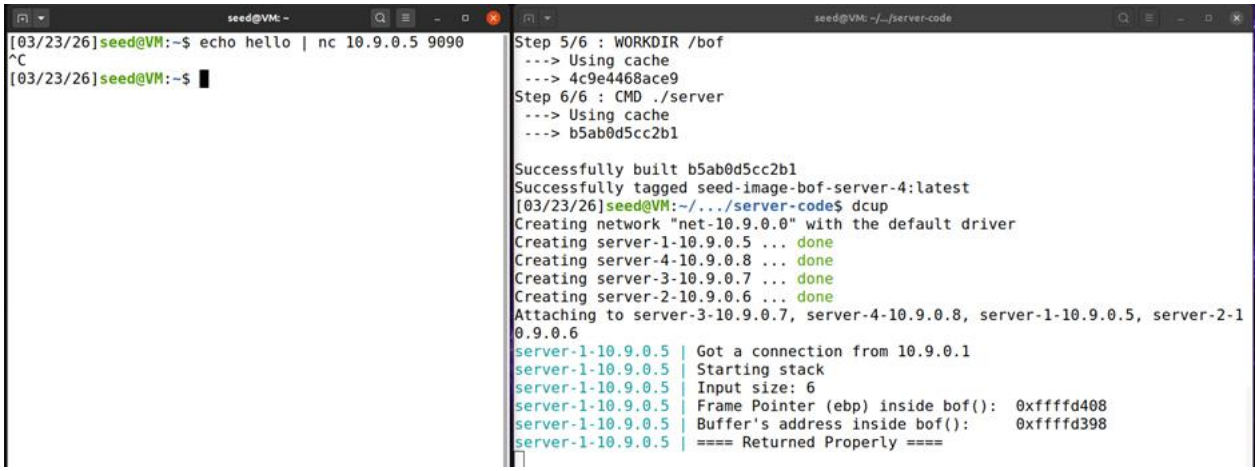


La première fenêtre correspond au serveur Docker en cours d'exécution. Elle reste **en écoute continue** sur le port 9090 et affiche toute nouvelle connexion entrante.

Lorsque la deuxième fenêtre exécute la commande :

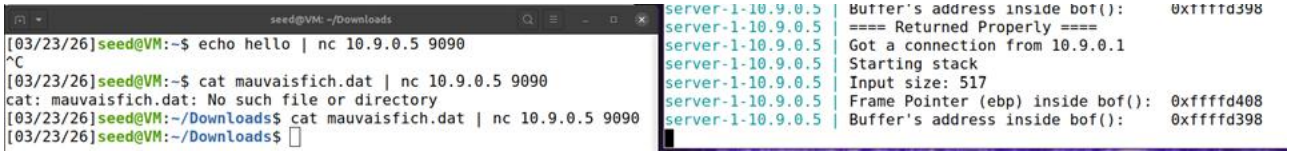
```
echo hello | nc 10.9.0.5 9090
```

le serveur de la première fenêtre **reçoit la connexion**, traite la chaîne envoyée et affiche les informations associées (début du traitement, taille de l'entrée, adresses mémoire, retour normal).



> Comme cette commande n'entraîne aucune erreur ni dépassement de tampon, la connexion se déroule normalement et se termine proprement. Le serveur reste actif et prêt à accepter d'autres connexions, tandis que la deuxième fenêtre termine simplement son envoi.
>
> Ainsi, les deux fenêtres demeurent actives : la première **écoute et réagit**, la seconde **établit une connexion ponctuelle puis se ferme normalement**.

6. Comportement du serveur avec mauvaisfich.dat

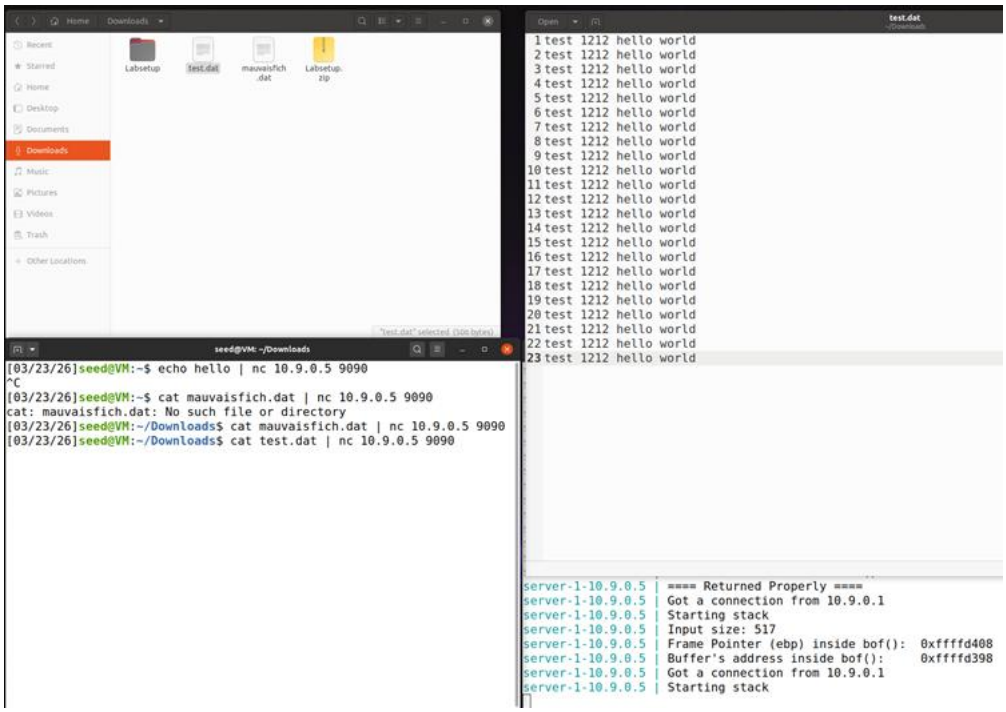


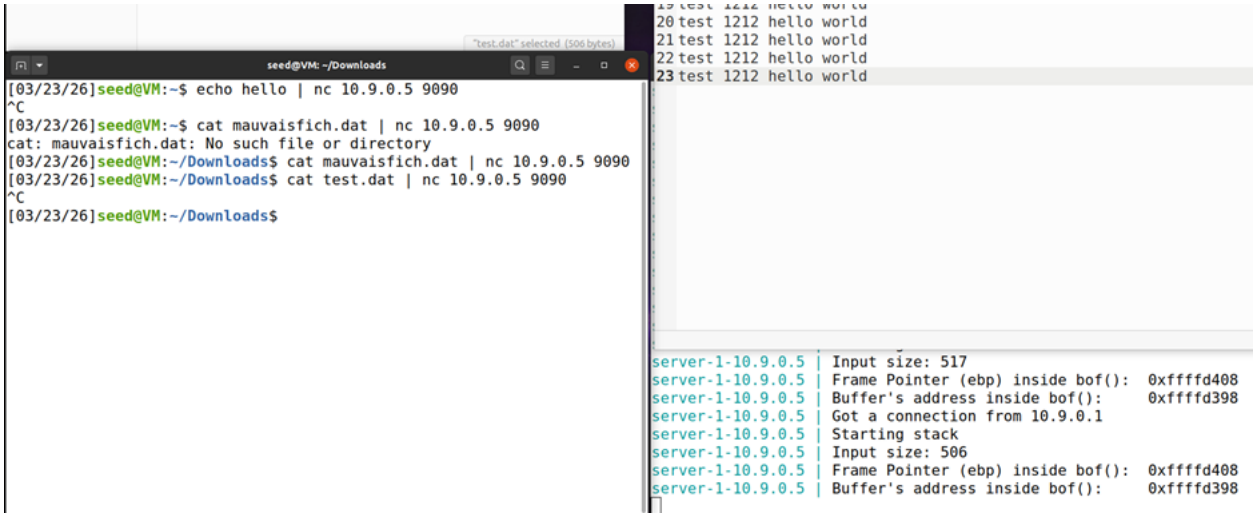
> Lorsque *mauvaisfich.dat* est envoyé, la première fenêtre (où le serveur Docker est en écoute) accepte la connexion, mais interrompt immédiatement la session après avoir reçu les données. La raison est que le fichier dépasse la taille maximale que le programme peut traiter sans erreur : le serveur détecte une entrée trop volumineuse et ferme la connexion pour préserver son fonctionnement.
>
> Dans la deuxième fenêtre, la commande se termine automatiquement, car c'est le serveur qui met fin à la connexion. Cela contraste avec la question 5, où la connexion restait ouverte et devait être interrompue manuellement par l'utilisateur.
>
> Ce comportement confirme que le fichier dépasse la limite interne du programme et déclenche une fermeture forcée de la connexion.

7. Critère déterminant du comportement

Comportement normal (attente sur « Starting stack »)

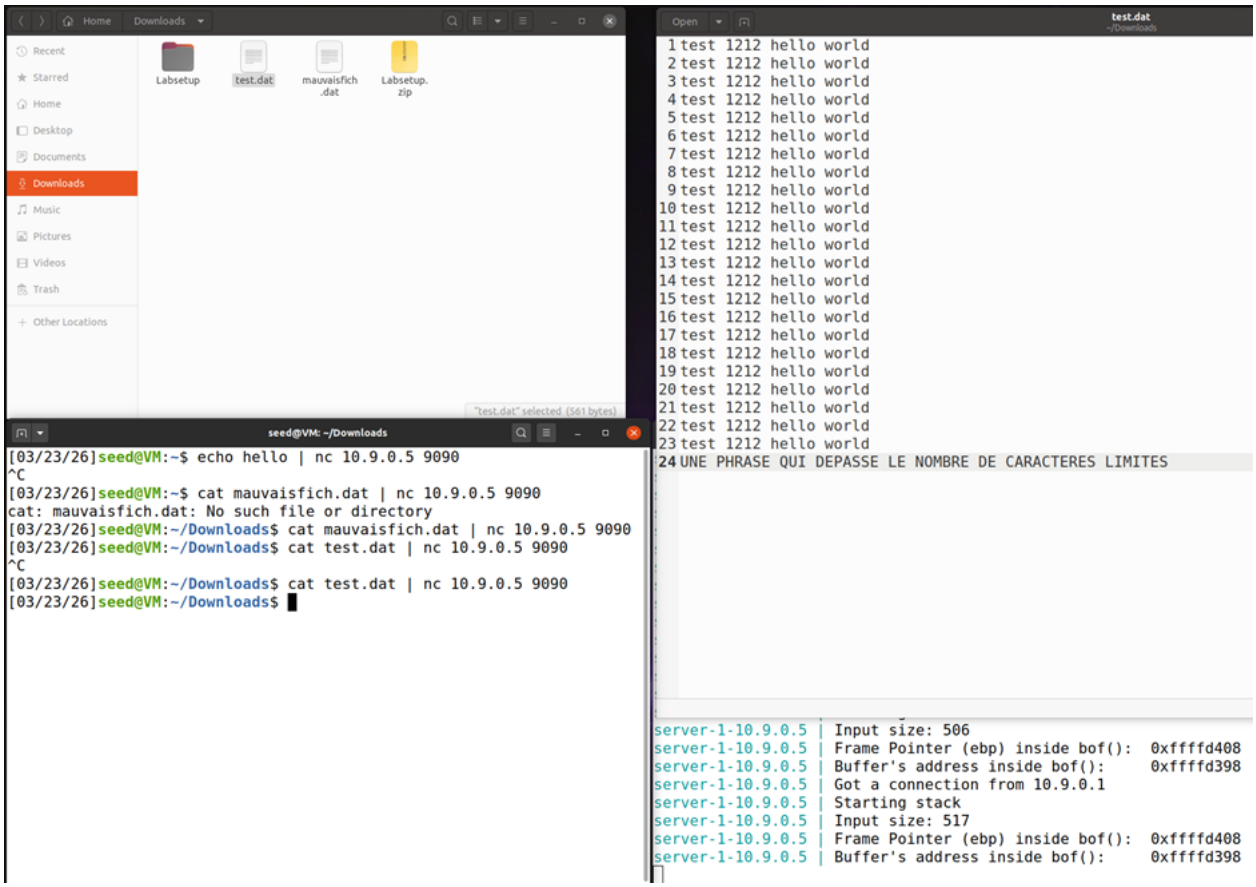
Le comportement du serveur dépend directement de la **taille des données envoyées**. Lors des tests, un comportement anormal apparaissait systématiquement lorsque la valeur affichée comme **"input size" atteignait ou dépassait 517 octets**





> Ce seuil correspond à la **limite maximale que le programme peut traiter sans provoquer une corruption de mémoire.** Tant que la taille du fichier restait inférieure à 517 caractères (par exemple 514), le serveur fonctionnait normalement : la connexion était établie et maintenue.

Comportement anormal (arrêt automatique)



> En revanche, dès que le fichier dépassait cette taille, le serveur rencontrait une condition anormale (probablement un dépassement de tampon), ce qui entraînait **une fermeture immédiate de la connexion**.

8. Exploitation offensive du débordement

> Il peut envoyer de larges fichiers aux serveurs attaqués pour faire déborder son buffer, faire crash son programme, écraser d'autres données, etc. Pour avoir un meilleur impact, le pirate devrait faire comme dans ce lab, et désactiver les contremesures avec des lignes de commandes. Un pirate peut exploiter ce comportement anormal pour transformer une simple erreur de dépassement de tampon en une vulnérabilité critique. Envoyer une charge utile de larges fichiers (payload) permettrait à l'attaquant de faire **déborder le buffer** de manière contrôlée, **d'écraser des zones mémoire sensibles**, comme l'adresse de retour, ou bien de **rediriger l'exécution du programme** vers du code malveillant qu'il aura injecté dans la pile ou ailleurs.
>
> Ce type d'attaque permettrait potentiellement d'exécuter du code arbitraire, d'obtenir un shell à distance, de prendre le contrôle complet du serveur, ou de provoquer un déni de service intentionnel.